Exam Summary

Module Name

Student

Last updated: 23. April 2026

Table of Contents

Week 1: Linear Algebra for Deep Learning

Chapter 1: Linear Algebra for Deep Learning

Part 1 — The Building Blocks

Type	Dims	Notation	Example
Scalar	0D	a	single loss value
Vector	1D	$\boldsymbol{x}$ — always column	$[1, 2, 3]^\top$
Matrix	2D	$\boldsymbol{A}$, element $A_{i,j}$	weight layer
Tensor	3D+	$\boldsymbol{A}$, element $A_{i,j,k}$	image batch

Column vector and matrix notation:

$$\boldsymbol{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \quad \boldsymbol{A} = \begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \\ A_{3,1} & A_{3,2} \end{pmatrix}$$

Part 2 — Operations

Matrix addition (same size only):

- *Used for:* combining two sets of features, adding residuals in ResNets

$$C_{i,j} = A_{i,j} + B_{i,j}$$

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} + \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 6 & 8 \\ 10 & 12 \end{pmatrix}$$

Scalar on matrix (a = scale, c = bias/shift):

- *Used for:* normalising data, applying a weight + bias to a whole layer

$$D_{i,j} = a \cdot B_{i,j} + c$$

$$2 \cdot \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} + 10 = \begin{pmatrix} 12 & 14 \\ 16 & 18 \end{pmatrix} \quad \rightarrow \text{every element } \times 2, \text{ then } +10$$

Broadcasting (vector added to every column of matrix):

- *Used for:* adding a bias vector to an entire batch of inputs at once

$$C_{i,j} = A_{i,j} + b_j$$

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} + \begin{pmatrix} 10 \\ 20 \end{pmatrix} = \begin{pmatrix} 11 & 22 \\ 13 & 24 \\ 15 & 26 \end{pmatrix}$$

Matrix multiplication:

- *Used for:* the core forward pass — transforming inputs through a weight layer: $\boldsymbol{y} = W\boldsymbol{x}$

$$\boldsymbol{C} = \boldsymbol{AB} \quad C_{i,j} = \sum_k A_{i,k} \cdot B_{k,j}$$

Shape: $(m \times k) \cdot (k \times n) = (m \times n)$ — inner dims must match.

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$

Inner / dot product (vectors only, result is a scalar):

- *Used for:* measuring similarity between two vectors; attention scores in Transformers

$$\boldsymbol{x}^\top \boldsymbol{y} = \sum_i x_i y_i = \|\boldsymbol{x}\|_2 \|\boldsymbol{y}\|_2 \cos\theta$$

$$\begin{pmatrix} 1 & 2 & 3 \end{pmatrix} \begin{pmatrix} 4 \\ 5 \\ 6 \end{pmatrix} = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 32$$

Hadamard (element-wise) product:

- *Used for:* gating mechanisms (LSTM, GRU), applying masks, dropout

$$\boldsymbol{C} = \boldsymbol{A} \odot \boldsymbol{B} \quad C_{i,j} = A_{i,j} \cdot B_{i,j}$$

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \odot \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 5 & 12 \\ 21 & 32 \end{pmatrix}$$

Properties of matrix multiplication:

- Distributive: $A(B + C) = AB + AC$
- Associative: $A(BC) = (AB)C$

Exam Trap: $AB \neq BA$ — matrix multiplication is **NOT commutative**. This is on every exam.

Transpose:

- *Used for:* converting column↔row vectors, computing dot products ($\boldsymbol{x}^\top \boldsymbol{y}$), flipping weight matrices in backpropagation

$$(A^\top)_{i,j} = A_{j,i} \quad (AB)^\top = B^\top A^\top$$

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}^\top = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix} \quad \rightarrow (2 \text{ times } 3) \text{ becomes } (3 \text{ times } 2)$$

Part 3 — Norms

- *Used for:* measuring magnitude of weights, gradients, errors; regularisation (L1/L2); comparing vectors

General L^p norm:

$$\|\boldsymbol{x}\|_p = \left(\sum_i |x_i|^p\right)^{\frac{1}{p}}, \quad p \geq 1$$

Example with $\boldsymbol{x} = [3, -4]^\top$:

$$\|\boldsymbol{x}\|_1 = 3 + 4 = 7 \quad \|\boldsymbol{x}\|_2 = \sqrt{9 + 16} = 5 \quad \|\boldsymbol{x}\|_\infty = 4$$

Norm	Formula	Use
L^2 (Euclidean)	$\|\boldsymbol{x}\|_2 = \sqrt{\sum_i x_i^2}$	most common
Squared L^2	$\|\boldsymbol{x}\|_2^2 = \boldsymbol{x}^\top \boldsymbol{x}$	easier to compute
L^1	$\|\boldsymbol{x}\|_1 = \sum_i \|x_i\|$	sparsity
L^0	count of non-zeros	**not a true norm**
L^∞	$\|\boldsymbol{x}\|_\infty = \max_i \|x_i\|$	max element
Frobenius	$\|A\|_F = \sqrt{\sum_{i,j} A_{i,j}^2} = \sqrt{\text{tr}(AA^\top)}$	matrix size

***Quick Reference* — A *vs* $A^\top$ *vs* A^{-1}**

Notation	Name	What it does	How it looks
A	Original matrix	Transforms input	unchanged
$A^\top$	Transpose	Swaps rows and columns	$(A^\top)_{i,j} = A_{j,i}$
A^{-1}	Inverse	Undoes A — like dividing	$A^{-1}A = I$

Starting with the same matrix A:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Transpose $A^\top$ — flip across diagonal (rows become columns):

$$A^\top = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix} \quad \rightarrow \text{row 1 } [1, 2] \text{ becomes col 1 } \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

Inverse A^{-1} — the "undo" matrix, found by calculation (not just flipping!):

$$A^{-1} = \begin{pmatrix} -2 & 1 \\ 3/2 & -1/2 \end{pmatrix} \quad \rightarrow A^{-1}A = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I$$

Special case — Orthogonal matrix: transpose = inverse ($A^\top = A^{-1}$)
→ only works because orthogonal matrices purely rotate (no stretching).

Exam Trap: $A^\top$ is always easy — just flip.
A^{-1} requires real computation — unless A is orthogonal (then $A^{-1} = A^\top$ for free).

Part 4 — Identity, Inverse, Solving Systems

- *Used for:* solving systems of equations $A\boldsymbol{x} = \boldsymbol{b}$; finding model parameters given data

$$A^{-1}A = I \quad A\boldsymbol{x} = \boldsymbol{b} \Rightarrow \boldsymbol{x} = A^{-1}\boldsymbol{b}$$

Example — solve $A\boldsymbol{x} = \boldsymbol{b}$:

$$A = \begin{pmatrix} 2 & 0 \\ 0 & 4 \end{pmatrix} \quad \boldsymbol{b} = \begin{pmatrix} 6 \\ 8 \end{pmatrix} \quad \rightarrow A^{-1} = \begin{pmatrix} 1/2 & 0 \\ 0 & 1/4 \end{pmatrix} \quad \rightarrow \boldsymbol{x} = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$$

How to compute A^{-1} — 2×2 formula (exam):

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \quad \Rightarrow \quad A^{-1} = \frac{1}{\det(A)} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} \quad \text{where } \det(A) = ad - bc$$

Steps: ① swap a and d on diagonal ② flip signs of b and c ③ divide by $\det(A)$

Example:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \rightarrow \det(A) = 1 \cdot 4 - 2 \cdot 3 = -2 \rightarrow A^{-1} = \frac{1}{-2} \begin{pmatrix} 4 & -2 \\ -3 & 1 \end{pmatrix} = \begin{pmatrix} -2 & 1 \\ 3/2 & -1/2 \end{pmatrix}$$

Verify: $A^{-1}A = \begin{pmatrix} -2 & 1 \\ 3/2 & -1/2 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I$✓

Exam Trap: If $\det(A) = ad - bc = 0$ → division by zero → **no inverse exists** (singular matrix).

For larger matrices: use Gauss-Jordan — write $[A \mid I]$, row-reduce until left side $= I$, right side becomes A^{-1}.

A^{-1} exists only if: A is square AND columns are linearly independent (non-singular).

Situation	Solutions
$m > n$ (overdetermined)	None (generically)
$m < n$ (underdetermined)	Infinitely many
$m = n$, independent cols	**Exactly one**

Part 5 — Special Matrices

Diagonal:

- *Used for:* efficient scaling; eigenvalue matrices in decompositions; fast inversion

$$\text{diag}(\boldsymbol{v})\boldsymbol{x} = \boldsymbol{v} \odot \boldsymbol{x} \quad \text{diag}\,(\boldsymbol{v})^{-1} = \text{diag}\left([1/v_1, ..., 1/v_n]^\top\right)$$

$$\text{diag}\left(\begin{pmatrix} 2 \\ 3 \\ 4 \end{pmatrix}\right)\boldsymbol{x} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 2x_1 \\ 3x_2 \\ 4x_3 \end{pmatrix} \quad \rightarrow \text{same as } \begin{pmatrix} 2 \\ 3 \\ 4 \end{pmatrix} \odot \boldsymbol{x}$$

Symmetric:

- *Used for:* covariance matrices, distance matrices, any matrix of pairwise values

$$A = A^\top$$

$$\begin{pmatrix} 1 & 2 & 3 \\ 2 & 5 & 4 \\ 3 & 4 & 6 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 5 & 4 \\ 3 & 4 & 6 \end{pmatrix}^\top \quad \rightarrow \text{mirrored across diagonal}$$

Orthogonal (rows AND cols are orthonormal):

- *Used for:* rotation matrices, V and U in SVD, efficient inversion (just transpose)

Exam Trap: Symmetric ≠ Orthogonal — completely different properties!
Symmetric: $A = A^\top$ (mirrored across diagonal).
Orthogonal: $A^\top A = I$ (columns perpendicular + unit length). A matrix can be orthogonal without looking symmetric at all.

How to check if a matrix is orthogonal — verify two conditions on every column:
1. **Unit length:** $\|\boldsymbol{c}_i\|_2 = 1$ — each column has length 1
2. **Perpendicular:** $\boldsymbol{c}_i^\top \boldsymbol{c}_j = 0$ for all $i \neq j$ — every pair of columns is at 90°

If both hold → orthogonal → use $A^{-1} = A^\top$ (free inverse, no computation).

Example — check col 1 and col 2 of $A = \frac{1}{3}\begin{pmatrix} 2 & -2 & 1 \\ 1 & 2 & 2 \\ 2 & 1 & -2 \end{pmatrix}$:

First, extract the columns (each column = one vertical slice of the matrix):

$$\boldsymbol{c}_1 = \frac{1}{3}\begin{pmatrix} 2 \\ 1 \\ 2 \end{pmatrix} \quad \boldsymbol{c}_2 = \frac{1}{3}\begin{pmatrix} -2 \\ 2 \\ 1 \end{pmatrix} \quad \boldsymbol{c}_3 = \frac{1}{3}\begin{pmatrix} 1 \\ 2 \\ -2 \end{pmatrix}$$

Check unit length — square each element, sum them, take the square root. Must equal 1:

$$\|\boldsymbol{c}_1\|_2 = \sqrt{\left(\frac{2}{3}\right)^2 + \left(\frac{1}{3}\right)^2 + \left(\frac{2}{3}\right)^2} = \sqrt{\frac{4}{9} + \frac{1}{9} + \frac{4}{9}} = \sqrt{\frac{9}{9}} = \sqrt{1} = 1\checkmark$$

Check perpendicular — dot product = multiply matching elements, then sum. Must equal 0:

$$\boldsymbol{c}_1^\top \boldsymbol{c}_2 = \frac{2}{3} \cdot \frac{-2}{3} + \frac{1}{3} \cdot \frac{2}{3} + \frac{2}{3} \cdot \frac{1}{3}$$

$$= \frac{-4}{9} + \frac{2}{9} + \frac{2}{9} = \frac{-4+2+2}{9} = \frac{0}{9} = 0\checkmark$$

Same check for every other pair: $\boldsymbol{c}_1^\top \boldsymbol{c}_3 = 0$ and $\boldsymbol{c}_2^\top \boldsymbol{c}_3 = 0$ (verify yourself!).

$$A^\top A = AA^\top = I \quad \Rightarrow \quad A^{-1} = A^\top$$

Inverting an orthogonal matrix = just transposing it — no computation needed.

Part 6 — Eigendecomposition

- *Used for:* understanding what a matrix does geometrically; PCA; finding principal directions of data; quadratic optimisation

$$A\boldsymbol{v} = \lambda \boldsymbol{v}$$

- $\boldsymbol{v}$ = eigenvector (direction unchanged by A, unit length $\|\boldsymbol{v}\| = 1$)

- λ = eigenvalue (scaling factor along that direction)

How to compute eigenvalues — solve the characteristic equation $\det(A - \lambda I) = 0$:

Step 1: Subtract λ from every diagonal element → get $A - \lambda I$

Step 2: Compute $\det(A - \lambda I) = 0$ → gives a polynomial in λ

Step 3: Solve for λ using the quadratic formula (for 2×2):

$$\lambda = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Example — find eigenvalues of $B = \begin{pmatrix} 3 & 12 \\ 12 & 56 \end{pmatrix}$:

$$\text{Step 1:} \quad B - \lambda I = \begin{pmatrix} 3-\lambda & 12 \\ 12 & 56-\lambda \end{pmatrix}$$

$$\text{Step 2:} \quad \det = (3-\lambda)(56-\lambda) - 12 \cdot 12 = \lambda^2 - 59\lambda + 24 = 0$$

$$\text{Step 3:} \quad \lambda = \frac{59 \pm \sqrt{59^2 - 4 \cdot 24}}{2} = \frac{59 \pm \sqrt{3385}}{2} \to \lambda_1 \approx 58.59, \quad \lambda_2 \approx 0.41$$

Example — $A = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$ has eigenvalues $\lambda_1 = 4, \lambda_2 = 2$:

$$A\begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 4 \\ 4 \end{pmatrix} = 4\begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \to \text{direction } [1,1]^\top \text{ scaled by } 4$$

$$A\begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 2 \\ -2 \end{pmatrix} = 2\begin{pmatrix} 1 \\ -1 \end{pmatrix} \quad \to \text{direction } [1,-1]^\top \text{ scaled by } 2$$

Full decomposition:

$$A = V\,\mathrm{diag}(\boldsymbol{\lambda})V^{-1} \quad V = \left[\boldsymbol{v}^{(1)}, ..., \boldsymbol{v}^{(n)}\right]$$

Intuition: **rotate** (by V) → **scale** (by λ) → **rotate back** (V^{-1}).

For real symmetric matrices: n real eigenvalues, orthogonal eigenvectors → $V^{-1} = V^\top$.

Optimization (exam!):

$$\max_{\|\boldsymbol{x}\|=1} \boldsymbol{x}^\top A\boldsymbol{x} \to \boldsymbol{x} = \text{eigenvec of max eigenvalue}$$

$$\min_{\|\boldsymbol{x}\|=1} \boldsymbol{x}^\top A\boldsymbol{x} \to \boldsymbol{x} = \text{eigenvec of min eigenvalue}$$

Part 7 — Singular Value Decomposition (SVD)

- *Used for:* pseudoinverse, data compression, dimensionality reduction (PCA), recommender systems, solving any linear system

Works for **any** matrix (generalization of eigendecomposition):

$$A = UDV^\top$$

- U: orthogonal — left singular vectors (output directions)

- D: diagonal — singular values (how much each direction is stretched)
- V: orthogonal — right singular vectors (input directions)

Full worked example — SVD of $A = \begin{pmatrix} 3 & 0 \\ 0 & 2 \\ 0 & 0 \end{pmatrix}$ (shape 3×2):

Step 1 — Compute $A^\top A$, find eigenvalues (used for both D and V):

$$A^\top A = \begin{pmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 0 & 2 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 9 & 0 \\ 0 & 4 \end{pmatrix} \rightarrow \lambda_1 = 9, \quad \lambda_2 = 4$$

Step 2 — Build D (square roots, same shape as A):

$$D = \begin{pmatrix} \sqrt{9} & 0 \\ 0 & \sqrt{4} \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 3 & 0 \\ 0 & 2 \\ 0 & 0 \end{pmatrix}$$

Step 3 — Build V (solve $(A^\top A - \lambda I)\boldsymbol{v} = 0$ for each λ, stack as columns):

$$\lambda_1 = 9 : \begin{pmatrix} 0 & 0 \\ 0 & -5 \end{pmatrix} \boldsymbol{v} = 0 \rightarrow \boldsymbol{v}^{(1)} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \lambda_2 = 4 : \begin{pmatrix} 5 & 0 \\ 0 & 0 \end{pmatrix} \boldsymbol{v} = 0 \rightarrow \boldsymbol{v}^{(2)} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$V = \left[\boldsymbol{v}^{(1)} \mid \boldsymbol{v}^{(2)} \right] = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Step 4 — Compute $AA^\top$, find eigenvectors (for U):

$$AA^\top = \begin{pmatrix} 3 & 0 \\ 0 & 2 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \end{pmatrix} = \begin{pmatrix} 9 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 0 \end{pmatrix} \rightarrow \lambda_1 = 9, \lambda_2 = 4, \lambda_3 = 0$$

$$\boldsymbol{u}^{(1)} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \quad \boldsymbol{u}^{(2)} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \quad \boldsymbol{u}^{(3)} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

$$U = \left[\boldsymbol{u}^{(1)} \mid \boldsymbol{u}^{(2)} \mid \boldsymbol{u}^{(3)} \right] = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Verify: $UDV^\top = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 0 & 2 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 3 & 0 \\ 0 & 2 \\ 0 & 0 \end{pmatrix} = A \checkmark$

Tip: Key insight: eigenvectors = directions. Stack them side by side as columns — col 1 = biggest λ, col 2 = next, and so on.

How to build U — eigenvectors of $AA^\top$, stacked as columns in descending λ order. Size: $m \times m$ (m = rows of A):

$$U = \left[\boldsymbol{u}^{(1)} \mid \boldsymbol{u}^{(2)} \mid \ldots \mid \boldsymbol{u}^{(m)} \right] \quad \text{col } i = \text{eigenvec of } AA^\top \text{ for } \lambda_i$$

Example — A is $3 \times 2 \rightarrow AA^\top$ is $3 \times 3 \rightarrow U$ is 3×3:

$$U = \begin{pmatrix} -0.283 & -0.868 & 0.408 \\ -0.538 & -0.208 & -0.816 \\ -0.794 & 0.451 & 0.408 \end{pmatrix}$$

How to build D — $\sqrt{\lambda_i}$ of $A^\top A$ on diagonal, descending order. Size = same shape as A:

$$D = \begin{pmatrix} \sqrt{\lambda_1} & 0 & \dots \\ 0 & \sqrt{\lambda_2} & \dots \\ \vdots & \vdots & \ddots \end{pmatrix} \quad \text{same shape as } A, \text{extra rows/cols} = 0$$

Example — A is 3×2, $\lambda_1 = 58.59$, $\lambda_2 = 0.41$:

$$D = \begin{pmatrix} \sqrt{58.59} & 0 \\ 0 & \sqrt{0.41} \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 7.65 & 0 \\ 0 & 0.64 \\ 0 & 0 \end{pmatrix}$$

How to build V — eigenvectors of $A^\top A$, stacked as columns in descending λ order. Size: $n \times n$ (n = cols of A):

$$V = \left[\boldsymbol{v}^{(1)} \mid \boldsymbol{v}^{(2)} \mid \dots \mid \boldsymbol{v}^{(n)}\right] \quad \text{col } i = \text{eigenvec of } A^\top A \text{ for } \lambda_i$$

Example — A has 2 cols $\rightarrow$ V is 2×2:

$$V = \begin{pmatrix} -0.211 & -0.977 \\ -0.977 & 0.211 \end{pmatrix}$$

Tip: Full SVD recipe for $A = UDV^\top$:
1. U: eigenvectors of $AA^\top$ as columns ($m \times m$)
2. D: $\sqrt{\lambda_i}$ on diagonal, same shape as A ($m \times n$)
3. V: eigenvectors of $A^\top A$ as columns ($n \times n$)

All in descending eigenvalue order.

Part 8 — Moore-Penrose Pseudoinverse

- *Used for:* least-squares regression, solving overdetermined systems (more equations than unknowns); any time a true inverse doesn't exist

When A^{-1} doesn't exist:

$$A^+ = VD^+U^\top \quad (D^+ : \text{reciprocal of each non-zero diagonal of D})$$

Example — $D = \begin{pmatrix} 3 & 0 \\ 0 & 2 \\ 0 & 0 \end{pmatrix} \rightarrow D^+ = \begin{pmatrix} 1/3 & 0 & 0 \\ 0 & 1/2 & 0 \end{pmatrix}$

Case	Result
More rows than cols (overdetermined)	Minimises $\|A\boldsymbol{x} - \boldsymbol{b}\|_2$ (least squares)
More cols than rows (underdetermined)	Minimises $\|\boldsymbol{x}\|_2$ among all solutions

Part 9 — Trace

- *Used for:* computing Frobenius norm, simplifying matrix derivative expressions, checking eigenvalue sums

$$\text{tr}(A) = \sum_i A_{i,i} = \sum_i \lambda_i$$

Example:

$$\text{tr}\left(\begin{pmatrix} 1 & 9 & 7 \\ 2 & 5 & 3 \\ 4 & 6 & 8 \end{pmatrix}\right) = 1 + 5 + 8 = 14$$

- $\text{tr}(A^\top) = \text{tr}(A)$
- $\text{tr}(ABC) = \text{tr}(CAB) = \text{tr}(BCA)$ ← cyclic invariant
- $\|A\|_F = \sqrt{\text{tr}(AA^\top)}$

Part 10 — Determinant

- *Used for:* checking if a matrix is invertible; measuring volume scaling; appears in probability density formulas (multivariate Gaussian)

$$\det(A) = \prod_i \lambda_i$$

Example:

$$\det\left(\begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}\right) = 3 \cdot 3 - 1 \cdot 1 = 8 \quad \rightarrow \lambda_1 \cdot \lambda_2 = 4 \cdot 2 = 8 \checkmark$$

- $\det(A) = 0 \rightarrow$ singular, not invertible (at least one eigenvalue is zero)
- $\det(A) \neq 0 \rightarrow$ invertible
- Geometric meaning: factor by which A scales volume

Cheat Sheet

WEEK 1 — LINEAR ALGEBRA CHEAT SHEET

Structures: scalar=0D · vector=1D(col) · matrix=2D · tensor=3D+

Multiply:
- Standard: $C_{i,j} = \sum_k A_{i,k} B_{k,j}$
- Dot product: $\boldsymbol{x}^\top \boldsymbol{y} = \|\boldsymbol{x}\| \|\boldsymbol{y}\| \cos\theta$
- Hadamard $\odot$: $C_{i,j} = A_{i,j} B_{i,j}$
- $AB \neq BA$ — NEVER commutative

Norms: $L^2 = \sqrt{\sum x^2}$ · $L^1 = \sum |x|$ · $L^\infty = \max|x|$ · $\|A\|_F = \sqrt{\text{tr}(AA^\top)}$

Special: Symmetric $A = A^\top$ · Orthogonal $A^{-1} = A^\top$ · Diagonal: invert = reciprocal diagonal

Eigen: $A\boldsymbol{v} = \lambda \boldsymbol{v} \rightarrow A = V \,\text{diag}(\lambda) V^{-1}$

SVD: $A = UDV^\top$ (always works)

Pseudoinverse: $A^+ = VD^+U^\top$

Trace: $\text{tr}(A) = \sum A_{i,i} = \sum \lambda_i$ (cyclic invariant)

Det: $\det(A) = \prod \lambda_i$ · zero = singular = not invertible

Week 2: Probability and Information Theory

Chapter 2: Probability and Information Theory

Part 1 — Random Variables

- *Used for:* describing uncertain quantities; modelling data distributions; defining loss functions

A **random variable** x can take on different values with associated probabilities.

Type	Values	Described by	Notation
Discrete	Countable set	Probability Mass Function (PMF)	$P(x = k)$
Continuous	Real interval	Probability Density Function (PDF)	$p(x)$

PMF (discrete) — probability at each exact value:

$$P(x = k) \geq 0 \quad \sum_k P(x = k) = 1$$

PDF (continuous) — probability density; integrate over an interval to get probability:

$$p(x) \geq 0 \quad \int_{\{-\infty\}}^{\{\infty\}} p(x)dx = 1 \quad P(a \leq x \leq b) = \int_a^b p(x)dx$$

Exam Trap: $p(x)$ is a **density**, not a probability. $p(x) > 1$ is valid! Only the integral over an interval gives probability.

Part 2 — Key Distributions

Bernoulli — single binary outcome ($x \in \{0, 1\}$):

- *Used for:* modelling coin flips, binary classification outputs, gates in LSTMs

$$P(x = 1) = \varphi, \quad P(x = 0) = 1 - \varphi$$

$$\mathbb{E}[x] = \varphi \quad \text{Var}(x) = \varphi(1 - \varphi)$$

Example: $\varphi = \frac{3}{4} \rightarrow \mathbb{E}[x] = \frac{3}{4}$, $\text{Var}(x) = \frac{3}{16}$.

Uniform (continuous) — equal probability over interval $[a, b]$:

- *Used for:* initialising weights, random sampling, baseline distributions

$$p(x) = \frac{1}{b - a} \quad x \in [a, b], \quad \text{else } 0$$

$$\mathbb{E}[x] = \frac{a + b}{2} \quad \text{Var}(x) = \frac{(b - a)^2}{12}$$

Derived from integration: $\mathbb{E}[x] = \int_a^b x \cdot \frac{1}{b-a} dx = \frac{a+b}{2}$ (the midpoint of $[a, b]$)

Example: $x \in [-0.5, 1] \rightarrow p(x) = \frac{2}{3}$, $\mathbb{E}[x] = \frac{1}{4}$, $\text{Var}(x) = \frac{3}{16}$.

Gaussian (Normal) — bell curve, most important distribution in ML:

- *Used for:* modelling real-valued data, noise, weight initialisation, latent spaces in VAEs

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

$$\mathbb{E}[x] = \mu \quad \text{Var}(x) = \sigma^2$$

Standard Normal: $\mu = 0$, $\sigma = 1$. Notation: $x \sim \mathcal{N}(\mu, \sigma^2)$.

Tip: Why Gaussian? The Central Limit Theorem: sums of many independent random variables converge to a Gaussian. Most real-world noise is approximately Gaussian.

Exponential — models time between events; heavy tail to the right:

$$p(x) = \lambda \exp(-\lambda x), \quad x \geq 0 \quad \mathbb{E}[x] = \frac{1}{\lambda}$$

Laplace — like Gaussian but with heavier tails; used in L1 regularisation:

$$p(x) = \frac{1}{2b} \exp\left(-\frac{|x-\mu|}{b}\right)$$

Part 3 — Expectations and Variance

- *Used for:* computing loss function values, quantifying spread, deriving gradient updates

Discrete: $\mathbb{E}[f(x)] = \sum_x f(x)P(x)$

Continuous: $\mathbb{E}[f(x)] = \int f(x)p(x)dx$

Variance: $\text{Var}(x) = \mathbb{E}[x^2] - (\mathbb{E}[x])^2 = \mathbb{E}[(x - \mathbb{E}[x])^2]$

Key properties:

- $\mathbb{E}[ax + b] = a, \mathbb{E}[x] + b$ — linearity
- $\text{Var}(ax) = a^2 \, \text{Var}(x)$
- $\text{Var}(x + y) = \text{Var}(x) + \text{Var}(y)$ only if x, y are independent

Covariance — measures linear dependence between two variables:

$$\text{Cov}(x, y) = \mathbb{E}[(x - \mathbb{E}[x])(y - \mathbb{E}[y])] = \mathbb{E}[xy] - \mathbb{E}[x]\mathbb{E}[y]$$

Part 4 — Bayes' Theorem

- *Used for:* updating beliefs given evidence; Naive Bayes classifiers; probabilistic inference in all Bayesian models

$$P(A \mid B) = \frac{P(B \mid A), P(A)}{P(B)}$$

- $P(A)$ — **prior**: belief before seeing evidence
- $P(B \mid A)$ — **likelihood**: probability of evidence given hypothesis
- $P(A \mid B)$ — **posterior**: updated belief after evidence
- $P(B)$ — **marginal** (normalisation constant)

Law of total probability (for computing $P(B)$):

$$P(B) = \sum_i P(B \mid A_i)P(A_i)$$

Example — cab problem: 5000 yellow cabs, 100 red cabs; witness accuracy 95%. Find $P(c = r \mid w = r)$:

$$P(c = r) = \frac{100}{5100} \approx 0.0196$$

$$P(w = r) = 0.95 \cdot \frac{100}{5100} + 0.05 \cdot \frac{5000}{5100} \approx 0.0676$$

$$P(c = r \mid w = r) = \frac{0.0196 \cdot 0.95}{0.0676} \approx 27.5\%$$

Exam Trap: Base rate neglect: even with a 95%-accurate witness, the answer is only 27.5%. Red cabs are so rare that most "red" reports are still wrong yellow cabs. Always account for the prior!

Bayesian classification — assign x to the class with highest posterior:

$$\text{assign to } c_i \quad \text{if} \quad P(y = c_i)p(x \mid y = c_i) > P(y = c_j)p(x \mid y = c_j)$$

The denominator $p(x)$ is the same for all classes and cancels.

Part 5 — Information Theory

- *Used for:* cross-entropy loss (classifier training), measuring model uncertainty, defining KL divergence for VAEs

Self-information — surprise of a single outcome:

$$I(x) = -\log P(x)$$

Rare events → large $I(x)$ (more surprising). Common events → small $I(x)$.

Log base: $\log_2$ = bits; ln = nats. Deep learning typically uses nats.

Shannon Entropy — expected surprise (average information content):

Discrete: $H(x) = -\sum_x P(x) \log P(x) = \mathbb{E}[-\log P(x)]$

Gaussian: $H(x) = \frac{1}{2}(\log(2\pi\sigma^2) + 1)$

Higher variance → higher entropy. Uniform distribution has maximum entropy.

Example:
- $\sigma_1^2 = 0.26 \to H(x \mid y = c_1) = 0.5(\ln(2\pi \cdot 0.26) + 1) \approx 0.745$ nats
- $\sigma_2^2 = 0.99 \to H(x \mid y = c_2) = 0.5(\ln(2\pi \cdot 0.99) + 1) \approx 1.414$ nats

Cross-Entropy — entropy of distribution P measured under model Q:

$$H(P, Q) = -\sum_x P(x) \log Q(x) = H(P) + D_{\mathrm{KL}}(P \parallel Q)$$

Used as the standard loss for classification (comparing true labels P to model predictions Q).

KL Divergence — measures how much Q differs from P:

$$D_{\mathrm{KL}}(P \parallel Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} \geq 0$$

Exam Trap: $D_{\mathrm{KL}}(P \parallel Q) \neq D_{\mathrm{KL}}(Q \parallel P)$ — KL divergence is **asymmetric**. It is not a distance metric!
- $D_{\mathrm{KL}}(P \| Q) = 0$ iff $P = Q$ everywhere.
- Minimising $H(P, Q)$ over Q is equivalent to minimising $D_{\mathrm{KL}}(P \| Q)$ (since $H(P)$ is constant).

Tip: Cross-entropy loss in practice:

$$\mathcal{L} = -\sum_k y_k \log \hat{y}_k$$

where y_k = true label (one-hot), $\hat{y}_k$ = softmax output. This is exactly $H(P, Q)$.

Cheat Sheet

WEEK 2 — PROBABILITY & INFORMATION THEORY CHEAT SHEET

Random Variables:
- Discrete → PMF: $\sum P(x) = 1$
- Continuous → PDF: $\int p(x)dx = 1$; $p(x)$ can exceed 1!

Key Distributions:
- Bernoulli(φ): $\mathbb{E} = \varphi$, $\mathrm{Var} = \varphi(1 - \varphi)$
- Uniform$[a, b]$: $p = \frac{1}{b-a}$, $\mathbb{E} = \frac{a+b}{2}$, $\mathrm{Var} = \frac{(b-a)^2}{12}$
- Gaussian(μ, σ^2): $p = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$

Expectation & Variance:
- $\mathbb{E}[f(x)] = \sum f(x)P(x)$ or $\int f(x)p(x)dx$
- $\mathrm{Var}(x) = \mathbb{E}[x^2] - (\mathbb{E}[x])^2$

Bayes: $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$ — always account for the prior!

Classification: assign to c_i if $P(c_i)p(x|c_i)$ is largest.

Entropy:

- Discrete: $H = -\sum P \log P$
- Gaussian: $H = \frac{1}{2}(\log(2\pi\sigma^2) + 1)$

Cross-entropy: $H(P, Q) = -\sum P \log Q = H(P) + D_{\mathrm{KL}}(P \| Q)$

KL-Divergence: $D_{\mathrm{KL}}(P \| Q) = \sum P \log \frac{P}{Q} \geq 0$; asymmetric!

Week 3: Numerical Computation

Chapter 3: Numerical Computation

Part 1 — General Methodology

- *Used for:* framing all of deep learning as an optimisation problem

Training a neural network = minimising the cross-entropy between the empirical data distribution $\hat{p}$ and the model distribution $p_{\text{model}(\boldsymbol{x};\boldsymbol{\theta})}$:

$$\hat{p}(\boldsymbol{x}) = \frac{1}{m}\sum_{i=1}^{m}\delta\left(\boldsymbol{x} - \boldsymbol{x}^{(i)}\right)$$

$$\min_{\boldsymbol{\theta}} H\left(\hat{p}_{\text{data}},, p_{\text{model}(\boldsymbol{\theta})}\right)$$

The optimal parameter vector: $\boldsymbol{x}^* = \arg\min f(\boldsymbol{x})$

Tip: Key insight: Minimising the cross-entropy $H(P, Q)$ is equivalent to minimising $D_{\text{KL}(P \parallel Q)}$, since $H(P)$ is constant with respect to the model parameters. Learning is fundamentally an optimisation problem.

Part 2 — Numerical Problems

- *Used for:* understanding why naive implementations of ML algorithms fail in practice

Computers store real numbers with finite precision — this causes three main classes of problems:

Problem	Cause	Effect
Quantization	Finite bits → discrete levels	Rounding errors accumulate
Underflow	Small values rounded to 0	Downstream $\frac{1}{x}$ or $\log(x)$ blows up
Overflow	Large values rounded to ∞	No further computation possible

Condition number — measures sensitivity of a function $f(\boldsymbol{x}) = \boldsymbol{A}^{-1}\boldsymbol{x}$ to small input errors:

$$\text{Condition number} = \max_{i,j}\left|\frac{\lambda_i}{\lambda_j}\right|$$

- $\approx 1 \rightarrow$ well conditioned (errors stay small)
- $\gg 1 \rightarrow$ ill conditioned (errors amplified, e.g. $> 10,000$)

Exam Trap: Poor conditioning is especially dangerous in linear regression: small measurement noise in $\boldsymbol{x}$ can produce wildly different parameter estimates $\hat{\boldsymbol{\beta}} = \boldsymbol{A}^{-1}\boldsymbol{x}$.

Part 3 — Softmax Stabilisation

- *Used for:* multi-class classification output layers; illustrates how to make numerically unstable functions safe

Softmax: $\text{softmax}(\boldsymbol{x})_i = \frac{\exp(x_i)}{\sum_{j=1}^{n}\exp(x_j)}$

Problem: if all inputs equal large positive $c \to \exp(c)$ overflows ($\frac{\infty}{\infty}$); large negative $c \to$ underflows ($\frac{0}{0}$).

Fix — subtract the maximum:

$$\boldsymbol{z} = \boldsymbol{x} - \max_i x_i$$

$$\text{softmax}(\boldsymbol{x})_i = \text{softmax}(\boldsymbol{z})_i \qquad \text{(identical output, numerically safe)}$$

Why it works:

- Largest entry of $\boldsymbol{z}$ is $0 \to \exp(0) = 1 \to$ denominator $> 1 \to$ no $\frac{0}{0}$
- All other entries negative $\to \exp < 1 \to$ no overflow

Exam Trap: log-softmax needs separate stabilisation: even a numerically stable softmax can underflow to 0, then $\log(0) = -\infty$. Modern frameworks (PyTorch, JAX) handle this automatically — but beware if computing by hand.

Part 4 — Gradient-Based Optimisation

- *Used for:* training every gradient-based ML model; the core algorithm of deep learning

Scalar case $y = f(x)$: derivative $f'(x) =$ slope. Moving in the direction of the slope decreases f:

$$f(x + \varepsilon) \approx f(x) + \varepsilon f'(x)$$

Algorithm — Gradient Descent:

1. Initialise at some point $\boldsymbol{x}^{(0)}$
2. Compute gradient $\nabla_{\boldsymbol{x}} f(\boldsymbol{x})$
3. Step in the **negative** gradient direction: $\boldsymbol{x}' = \boldsymbol{x} - \varepsilon \nabla_{\boldsymbol{x}} f(\boldsymbol{x})$
4. Repeat until $\nabla_{\boldsymbol{x}} f(\boldsymbol{x}) = \boldsymbol{0}$

Critical points — where $f'(x) = 0$ (gradient descent halts):

Type	Description
Local minimum	Both directions go up
Local maximum	Both directions go down

Saddle point	$f'(x) = 0$ but neither min nor max

Tip: In deep learning, we never find the **global** minimum — the loss landscape has millions of saddle points and local minima. The goal is to be "good enough": a local minimum with low enough loss value.

Part 5 — Gradient in Multiple Dimensions

- *Used for:* optimising neural network parameters $\boldsymbol{\theta}$ which are always high-dimensional vectors

For $f : \mathbb{R}^n \to \mathbb{R}$ (vector input, scalar output):

Partial derivative: slope of f when changing x_i only:

$$\frac{\partial}{\partial x_i} f(\boldsymbol{x})$$

Gradient: vector of all partial derivatives (same size as $\boldsymbol{x}$):

$$\nabla_{\boldsymbol{x}} f(\boldsymbol{x}) = \begin{pmatrix} \frac{\partial f(\boldsymbol{x})}{\partial x_1} \\ \frac{\partial f(\boldsymbol{x})}{\partial x_2} \\ \vdots \\ \frac{\partial f(\boldsymbol{x})}{\partial x_n} \end{pmatrix}$$

Directional derivative in unit direction $\boldsymbol{u}$:

$$\frac{\partial}{\partial \alpha} f(\boldsymbol{x} + \alpha \boldsymbol{u}) = \boldsymbol{u}^T \nabla_{\boldsymbol{x}} f(\boldsymbol{x}) \quad \text{at } \alpha = 0$$

The direction that **minimises** the directional derivative is the **negative gradient** direction (where $\cos\theta = -1$, i.e. $\theta = 180°$):

Gradient descent update rule:

$$\boldsymbol{x}' = \boldsymbol{x} - \varepsilon \nabla_{\boldsymbol{x}} f(\boldsymbol{x})$$

- ε too small → learning takes forever
- ε too large → chance of stepping over a minimum

Part 6 — Jacobian Matrices

- *Used for:* backpropagation through vector-to-vector layers; computing gradients of the softmax

For functions $f : \mathbb{R}^m \to \mathbb{R}^n$ (vector input **and** vector output):

Jacobian matrix $\boldsymbol{J} \in \mathbb{R}^{n \times m}$:

$$J_{i,j} = \frac{\partial}{\partial x_j} f(\boldsymbol{x})_i$$

Example for $n = m = 2$:

$$\boldsymbol{J} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \end{pmatrix}$$

Tip: The Jacobian is the generalisation of the gradient to vector-valued functions. Row i of $\boldsymbol{J}$ = gradient of output $f_{i(\boldsymbol{x})}$.

Part 7 — Second Derivatives and the Hessian

- *Used for:* Newton's method; understanding curvature; second-order optimisation; diagnosing saddle points

Curvature is the slope of the slope:

Curvature	Slope behaviour	GD step quality
Negative ($f'' < 0$)	Slope gets smaller	Too small (pessimistic)
Zero ($f'' = 0$)	Slope stays constant	Just right
Positive ($f'' > 0$)	Slope gets larger	Too large (optimistic)

Hessian matrix — matrix of all second-order partial derivatives:

$$\boldsymbol{H}(f)(\boldsymbol{x})_{i,j} = \frac{\partial^2}{\partial x_i \partial x_j} f(\boldsymbol{x})$$

Example for $n = 2$:

$$\boldsymbol{H} = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} \end{pmatrix}$$

Symmetry: $H_{i,j} = H_{j,i}$ → eigendecomposition with real eigenvalues: $\boldsymbol{H} = \boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^T$

Directional second derivative in direction $\boldsymbol{d}$: $\boldsymbol{d}^T\boldsymbol{H}\boldsymbol{d}$

Curvature in direction $\boldsymbol{d}$ is a weighted sum of eigenvalues:

$$\boldsymbol{d}^T\boldsymbol{H}\boldsymbol{d} = \sum_i (\cos\theta_i)^2 \lambda_i$$

- Maximum curvature = $\lambda_{\max}$ in direction of $\boldsymbol{v}_{\max}$
- Minimum curvature = $\lambda_{\min}$ in direction of $\boldsymbol{v}_{\min}$

Part 8 — Taylor Approximation and Second Derivative Test

- *Used for:* Newton's method; understanding when gradient descent overshoots; classifying critical points

2nd-order Taylor approximation:

$$f(\boldsymbol{x}) \approx f\left(\boldsymbol{x}^{(0)}\right) + \left(\boldsymbol{x} - \boldsymbol{x}^{(0)}\right)^T \boldsymbol{g} + \frac{1}{2}\left(\boldsymbol{x} - \boldsymbol{x}^{(0)}\right)^T \boldsymbol{H}\left(\boldsymbol{x} - \boldsymbol{x}^{(0)}\right)$$

where $\boldsymbol{g} = \nabla f\left(\boldsymbol{x}^{(0)}\right)$ and $\boldsymbol{H} = \boldsymbol{H}(f)\left(\boldsymbol{x}^{(0)}\right)$.

After one gradient descent step $\boldsymbol{x}^{(0)} - \varepsilon \boldsymbol{g}$:

$$f\left(\boldsymbol{x}^{(0)} - \varepsilon \boldsymbol{g}\right) \approx f\left(\boldsymbol{x}^{(0)}\right) - \varepsilon \boldsymbol{g}^T \boldsymbol{g} + \frac{1}{2}\varepsilon^2 \boldsymbol{g}^T \boldsymbol{H} \boldsymbol{g}$$

The third term $\frac{1}{2}\varepsilon^2 \boldsymbol{g}^T \boldsymbol{H} \boldsymbol{g}$ governs whether the step actually decreases f:

- $\boldsymbol{g}^T \boldsymbol{H} \boldsymbol{g} \leq 0 \rightarrow$ larger ε always helps
- $\boldsymbol{g}^T \boldsymbol{H} \boldsymbol{g} > 0 \rightarrow$ strong curvature can **increase** f; must use small ε

Second derivative test:

Critical point type	1D: $f''(x)$	nD: Eigenvalues of $\boldsymbol{H}$
Local minimum	> 0	All positive
Local maximum	< 0	All negative
Saddle point	$= 0$	Mixed (some $+$, some $-$)

Newton's method — uses the Hessian to take a single optimal step to the critical point:

$$\boldsymbol{x}^{(i)} = \boldsymbol{x}^{(i-1)} - \boldsymbol{H}(f)\left(\boldsymbol{x}^{(i-1)}\right)^{-1} \nabla_{\boldsymbol{x}} f\left(\boldsymbol{x}^{(i-1)}\right)$$

Exam Trap: Newton's method converges to **any** critical point — including saddle points! In high dimensions (1M parameters), even if 999′999 eigenvalues are positive and just 1 is negative, the point is still a saddle — not a minimum. Steepest descent often works better in practice.

Part 9 — Condition Number and Poor Conditioning

- *Used for:* diagnosing slow convergence; motivating adaptive learning rate methods

$$\text{Condition number of Hessian} = \max_{i,j} \left| \frac{\lambda_i}{\lambda_j} \right|$$

Condition number	Curvature	Effect on GD

Small (≈ 1)	Similar in all directions	Large ε fine, fast convergence
Large ($\gg 1$)	High in one direction, low in another	Must use small $\varepsilon \rightarrow$ zigzag, slow progress

The loss landscape with a high condition number looks like an elongated ellipse — gradient descent zigzags along the steep direction while making tiny progress in the flat direction. This motivates adaptive optimisers (Adam, RMSProp) to be discussed later.

Cheat Sheet

WEEK 3 — NUMERICAL COMPUTATION CHEAT SHEET

Numerical problems:
- Underflow: small values $\rightarrow 0$ (triggers division/log errors)
- Overflow: large values $\rightarrow \infty$
- Softmax fix: subtract max before applying exp
- Condition number $= \max|\frac{\lambda_i}{\lambda_j}|$; large $\rightarrow$ ill conditioned

Gradient descent:

$$\boldsymbol{x}' = \boldsymbol{x} - \varepsilon \nabla_{\boldsymbol{x}} f(\boldsymbol{x})$$

- Gradient points in direction of steepest **ascent**
- Negative gradient = steepest **descent**
- Terminates at critical points where $\nabla_{\boldsymbol{x}} f = \mathbf{0}$

Partial / directional derivatives:
- Gradient: $\nabla_{\boldsymbol{x}} f$ — vector of all partials
- Directional: $\frac{\partial}{\partial \alpha} f(\boldsymbol{x} + \alpha \boldsymbol{u}) = \boldsymbol{u}^T \nabla_{\boldsymbol{x}} f(\boldsymbol{x})$

Jacobian ($f : \mathbb{R}^m \rightarrow \mathbb{R}^n$): $J_{i,j} = \frac{\partial}{\partial x_j} f(\boldsymbol{x})_i \in \mathbb{R}^{n \times m}$

Hessian ($f : \mathbb{R}^n \rightarrow \mathbb{R}$): $H_{i,j} = \frac{\partial^2}{\partial x_i \partial x_j} f$ — symmetric, real eigenvalues

2nd order Taylor:

$$f(\boldsymbol{x}) \approx f\left(\boldsymbol{x}^{(0)}\right) + \left(\boldsymbol{x} - \boldsymbol{x}^{(0)}\right)^T \boldsymbol{g} + \frac{1}{2}\left(\boldsymbol{x} - \boldsymbol{x}^{(0)}\right)^T \boldsymbol{H}\left(\boldsymbol{x} - \boldsymbol{x}^{(0)}\right)$$

Second derivative test:
- Min: $f'' > 0$ / all $\lambda > 0$
- Max: $f'' < 0$ / all $\lambda < 0$
- Saddle: $f'' = 0$ / mixed λ

Newton's method:

$$\boldsymbol{x}^{(i)} = \boldsymbol{x}^{(i-1)} - \boldsymbol{H}^{-1} \nabla f$$

— fast but attracted to saddle points!

Week 4

Chapter 4: Machine Learning Basics (Part 1)

Definition & Grundkonzepte

Tip: Die goldene ML-Definition: Ein ML-Algorithmus verbessert die **Performance P** eines Computers bei einer **Aufgabe T** durch **Erfahrung E**.

- **T** (Task): z.B. $\hat{y}$ aus $\boldsymbol{x}$ vorhersagen
- **P** (Performance): z.B. MSE auf Testdaten
- **E** (Experience): Trainingsdaten

Lineare Regression

Lineares Modell mit Bias-Term:

$$\hat{y} = \boldsymbol{w}^T \boldsymbol{x} + b$$

Bias-Term-Trick — füge Spalte von 1en zu $\boldsymbol{X}$ hinzu:

$$\boldsymbol{X} = \begin{pmatrix} x_{1,1} & \dots & 1 \\ x_{2,1} & \dots & 1 \\ \vdots & & \vdots \end{pmatrix}$$

Dann: $\hat{y} = \boldsymbol{w}^T \boldsymbol{x}$ (kein separates b nötig)

Mean Squared Error:

$$\text{MSE}_{\text{train}} = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

Analytische Lösung (nur für lin. Regression möglich!):

$$\hat{\boldsymbol{w}} = \arg\min_{\boldsymbol{w}} \text{MSE}_{\text{train}} \quad \rightarrow \quad \nabla_{\boldsymbol{w}} \text{MSE} = 0 \text{ lösen}$$

Exam Trap: Prüfungsfalle: Der **Bias-Term** b im Modell $\hat{y} = \boldsymbol{w}^T \boldsymbol{x} + b$ ist **nicht dasselbe** wie der statistische **Bias** eines Schätzers! Zwei völlig verschiedene Konzepte mit demselben Wort.

Capacity, Overfitting & Underfitting

i.i.d.-Annahmen (Independent & Identically Distributed):

- Datenpunkte sind **unabhängig** voneinander
- Training- und Testdaten kommen aus **derselben** Verteilung p_{data}

Folgerung:

$$\mathbb{E}[\text{MSE}_{\text{train}}] = \mathbb{E}[\text{MSE}_{\text{test}}]$$

Ziel der Optimierung — zwei Bedingungen:

1. $\text{MSE}_{\text{train}}$ klein
2. $\text{Gap} = \text{MSE}_{\text{test}} - \text{MSE}_{\text{train}}$ klein

Tip: Capacity = „Flexibilität“ des Modells

| Situation | Training-Fehler | Test-Fehler | Problem | |—|—|—|—| | Zu kleine Capacity | groß | groß | **Underfitting** | | Passende Capacity | klein | klein | ✓ Optimal | | Zu große Capacity | ≈ 0 | sehr groß | **Overfitting** |

Faustregel: Training-Fehler sinkt **immer** mit Capacity. Test-Fehler hat U-Form!

Beispiel Polynome: Linear ($n = 2$ Param.) → Quadratisch ($n = 3$) → Grad 9 ($n = 10$)

Exam Trap: Bayes Error / Irreducible Error: Selbst ein perfekter Klassifikator hat $\text{MSE}_{\text{test}} > 0$, weil Messfehler, fehlende Kausalzusammenhänge etc. unvermeidbar sind. Man kann den Bayes Error **nicht** durch bessere Modelle reduzieren!

No Free Lunch Theorem

Tip: No Free Lunch Theorem (gemittelt über alle möglichen Datenverteilungen): Kein ML-Algorithmus ist universell besser als ein anderer.

Konsequenz (positiv!): Wir müssen unseren Algorithmus auf unser spezifisches Problem zuschneiden — dann können wir den optimalen Algorithmus finden.

Regularisierung

Definition Regularisierung: Jede Modifikation des Lernalgorithmus, die den **Generalisierungsfehler** reduziert, aber **nicht** den Trainingsfehler.

Beispiel: Weight Decay (L2-Regularisierung):

$$\hat{\boldsymbol{w}} = \arg\min_{\boldsymbol{w}} [\text{MSE}_{\text{train}} + \lambda \boldsymbol{w}^T \boldsymbol{w}]$$

| λ | Capacity | Effekt | |—|—|—| | 0 | hoch | keine Regularisierung | | $\in (0, \infty)$ | mittel | kontrollierte Flexibilität | | $\to \infty$ | ≈ 0 | $\boldsymbol{w}^T \boldsymbol{w} \to 0$ erzwungen |

Tip: DL-Philosophie:
1. Baue ein sehr großes, flexibles Modell (hohe Capacity)
2. Regularisiere es gezielt

→ In der Praxis erfolgreicher als Capacity direkt zu optimieren!

Hyperparameter & Validation Sets

Dataset-Split (typisch):

$$\text{Full Dataset} = \underbrace{60\%}_{\text{Training}} + \underbrace{20\%}_{\text{Validation}} + \underbrace{20\%}_{\text{Test}}$$

Verwendung:

- **Training Set**: Lernen der Parameter $\boldsymbol{\theta}$
- **Validation Set**: Wahl der Hyperparameter (z.B. λ, Polynomgrad)
- **Test Set**: Finale Schätzung des Generalisierungsfehlers

Exam Trap: Prüfungsfalle — Hyperparameter-Selektion:

- Training Set für Hyperparameter? → **Nein!** → wählt immer höchste Capacity
- Test Set für Hyperparameter? → **Nein!** → Test Set darf **nie** für Entscheidungen benutzt werden — nur zur finalen Evaluation!
- **Lösung:** Immer Validation Set verwenden!

Schätzer, Bias & Varianz

Point Estimator:

$$\hat{\boldsymbol{\theta}}_m = \boldsymbol{g}\left(x^{(1)}, ..., x^{(m)}\right)$$

basierend auf m i.i.d. Datenpunkten.

Bias eines Schätzers:

$$\text{bias}\left(\hat{\theta}_m\right) = \mathbb{E}\left[\hat{\theta}_m\right] - \theta$$

Unverzerrt (unbiased): $\mathbb{E}\left[\hat{\theta}_m\right] = \theta$

Asymptotisch unverzerrt: $\lim_{m\to\infty} \mathbb{E}\left[\hat{\theta}_m\right] = \theta$

Varianz und Standardfehler:

$$\text{Var}\left(\hat{\theta}_m\right) = \mathbb{E}\left[\left(\hat{\theta}_m - \mathbb{E}\left[\hat{\theta}_m\right]\right)^2\right]$$

$$\text{SE}\left(\hat{\theta}_m\right) = \sqrt{\text{Var}\left(\hat{\theta}_m\right)}$$

MSE eines Schätzers (Bias-Varianz-Zerlegung):

$$\text{MSE}\left(\hat{\theta}_m\right) = \text{Bias}\left(\hat{\theta}_m\right)^2 + \text{Var}\left(\hat{\theta}_m\right)$$

Wichtige Schätzer-Beispiele:

Bernoulli-Verteilung ($P(x = 1) = \theta$):

$$\hat{\theta}_m = \frac{1}{m}\sum_{i=1}^{m} x^{(i)} \quad \rightarrow \quad \text{unbiased!} \quad \text{Bias} = 0$$

Gauß-Verteilung — Mittelwert ($\mathbb{E}[x] = \mu$):

$$\hat{\mu}_m = \frac{1}{m}\sum_{i=1}^{m} x^{(i)} \quad \rightarrow \quad \text{unbiased!}$$

Gauß-Verteilung — Varianz (naive Schätzung):

$$\hat{\sigma}_m^2 = \frac{1}{m}\sum_{i=1}^{m}\left(x^{(i)} - \hat{\mu}_m\right)^2 \quad \rightarrow \quad \text{BIASED!}$$

$$\text{bias}(\hat{\sigma}_m^2) = -\frac{\sigma^2}{m} \quad \left(= -\frac{m-1}{m}\sigma^2 - \sigma^2 ...\right)$$

Unbiased Sample Variance:

$$\hat{\sigma}_m^2 = \frac{1}{m-1}\sum_{i=1}^{m}\left(x^{(i)} - \hat{\mu}_m\right)^2 \quad \rightarrow \quad \text{unbiased!}$$

Standardfehler des Mittelwerts:

$$\text{SE}(\hat{\mu}_m) = \frac{\sigma}{\sqrt{m}}$$

Tip: Konsistenz eines Schätzers:

$$\hat{\theta}_m \xrightarrow{p} \theta \quad \text{für} \quad m \rightarrow \infty$$

Das bedeutet: (asymptotisch) unverzerrt + Standardfehler $\rightarrow 0$

Merkhilfe Standardfehler: $\text{SE} = \frac{\sigma}{\sqrt{m}}$ $\rightarrow$ **mehr Daten = kleinerer Fehler** ($\sqrt{m}$-Gesetz)

95%-Konfidenzintervall:

$$\text{CI} = [\mu - 2\sigma, \mu + 2\sigma]$$

Bias-Varianz Trade-Off

Zerlegung des Generalisierungsfehlers:

$$\underbrace{\text{MSE}\left(\hat{\theta}_m\right)}_{\text{Generalisierungsfehler}} = \underbrace{\text{Bias}\left(\hat{\theta}_m\right)^2}_{\text{Systematischer Fehler}} + \underbrace{\text{Var}\left(\hat{\theta}_m\right)}_{\text{Zufälliger Fehler}}$$

Tip: Trade-Off-Tabelle:

Capacity	Bias	Varianz	Problem
niedrig	**hoch**	niedrig	Underfitting
mittel	mittel	mittel	✓ Optimal
hoch	niedrig	**hoch**	Overfitting

Ziel: Mittlere Capacity mit optimalem Bias-Varianz-Gleichgewicht finden — typischerweise via Validation Set und Regularisierung.

Übungs-Cheat-Sheet

Schnellantworten für Multiple Choice:

Ziel ML: Kleiner Test-/Generalisierungsfehler (nicht Trainingsfehler!)

Overfitting: großer Test-Fehler + kleiner Train-Fehler → Capacity ↓ oder Daten ↑ oder Regularisierung ↑

Underfitting: großer Test- UND Train-Fehler → Capacity ↑

Overfitting tritt auf wenn:
- Trainingsdaten klein
- Regularisierungsterm klein (λ klein)
- Trainingsfehler < Bayes Error

Regularisierung reduziert: Generalisierungsfehler + Test-Fehler (aber **nicht** Training-Fehler direkt)

Dataset-Nutzung:
- Training → Parameter lernen
- Validation → Hyperparameter wählen
- Test → finale Evaluation (nur einmal!)

Unbiased Variance: Teile durch $(m-1)$, nicht m!

Bias-Korrektur für |x|-Schätzer auf $[-0.8, 1.2]$:

$$\text{bias} = \mathbb{E}[\![x]\!] - \mu = 0.52 - 0.2 = 0.32$$

Korrigierter Schätzer: $\hat{\mu} = \frac{1}{m}\sum|x^{(i)}| - 0.32$

Unbiased Max-Schätzer für Uniform$[0, \theta]$:

$$\hat{\theta} = \frac{m+1}{m} \cdot \max\left(x^{(1)}, ..., x^{(m)}\right)$$